COMS4995W31
Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Spring 2026

Rewards & Retrieval for LLM

AI of the Week: RL for LLM

🧠 MIT "TLT": 2x faster RL training for reasoning LLMs

- Researchers use idle GPU time during RL training to co-train a small "drafter" model on the fly, accelerating reasoning LLM training by 70–210%
- Key insight: fast decoding adapted for RL, where the policy model stays aligned with the continuously updating

🤖 DeepMind AlphaEvolve: LLM rewrites its own RL algorithms

- An LLM-powered evolutionary agent that automatically discovers new Multi-Agent RL algorithm variants, outperforming hand-designed baselines
- Applied to game theory - replaces manual algorithm design with automated search

Agenda

- Motivation
- Data for RL
- RL Algorithms for LLMs
- Retrieval Augmented Generation

Motivation

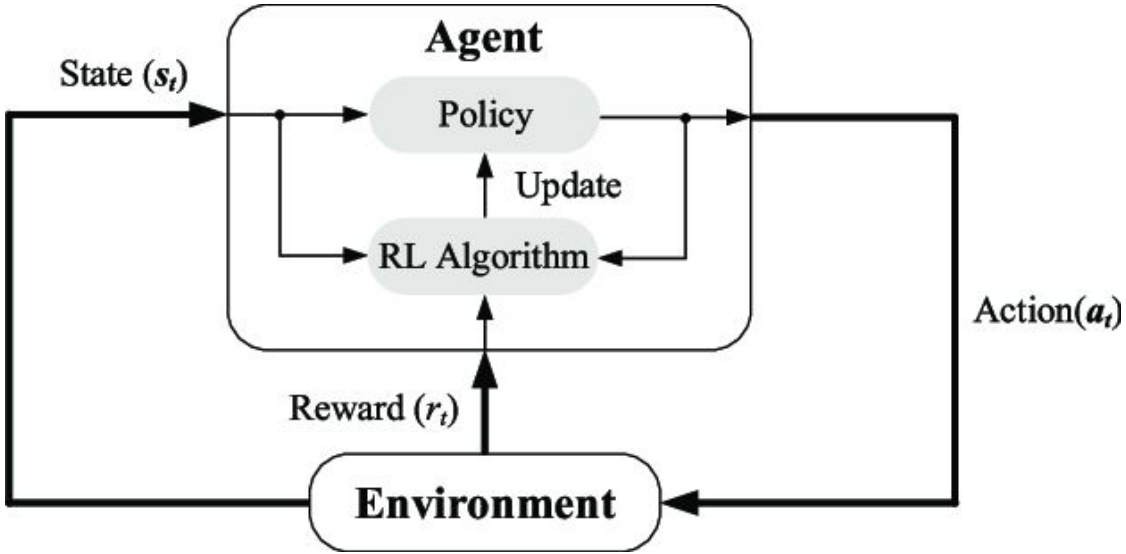
What is Reinforcement Learning (RL)? 🧭

Reinforcement Learning (RL) is a paradigm where an agent interacts with an environment, takes actions, and receives rewards; over time it learns a policy model that **maximizes expected cumulative reward**

Unlike SFT (imitate labeled examples), RL learns by doing

- tries things
- sees consequences
- updates behavior accordingly

What is Reinforcement Learning (RL)? 🧭



What is Reinforcement Learning (RL)? 🧭

Core loop:

1. The agent observes a state s
2. Chooses an action a based on its policy $\pi(a|s)$
3. Receives a reward r and new state s'
4. Updates its policy π to maximize expected future rewards

What is Reinforcement Learning (RL)? 🧭

Goal: Find a policy π* that maximizes

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where γ is the discount factor (balances short-term vs long-term gains)

Robot Example 🤖

A robot is learning to navigate a maze

- r_t = immediate feedback the agent receives at time t
- At the goal: r_t = +1 → a strong positive signal for success
- Each step: r_t = −0.01 → a small penalty encouraging efficiency
- Goal: reach the goal quickly (avoid many −0.01 steps)

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

γ=0.9 → earlier rewards still valuable

γ=0.1 → only immediate rewards count

Why RL for LLMs? 🚀

LLMs first learn by imitation:

- Pre-training: predict the next token from massive text corpora
- Supervised Fine-Tuning: imitate high-quality human answers

Both stages teach models to sound right, not necessarily be right

They imitate human patterns but do not optimize for deeper goals

Reinforcement Learning introduces explicit objectives through rewards

→ to optimize for quality, usefulness, safety and more

What Objectives is LLM Optimizing For? 🧭

Helpfulness - useful, relevant, stays on task, concise when needed

Honesty - factually grounded, transparent about uncertainty

Harmlessness - avoids unsafe, biased, or toxic content

Reasoning - correct intermediate logic, verifiable steps

Efficiency - answers quickly and clearly, minimal redundancy

Creativity - generates diverse, novel yet relevant ideas

How RL Maps to LLMs 🤖

Agent = the language model π itself

It observes a prompt (state) and decides what to say next

Action = each generates a series of tokens

Each token choice changes the conversation or reasoning trajectory

Environment = the task context

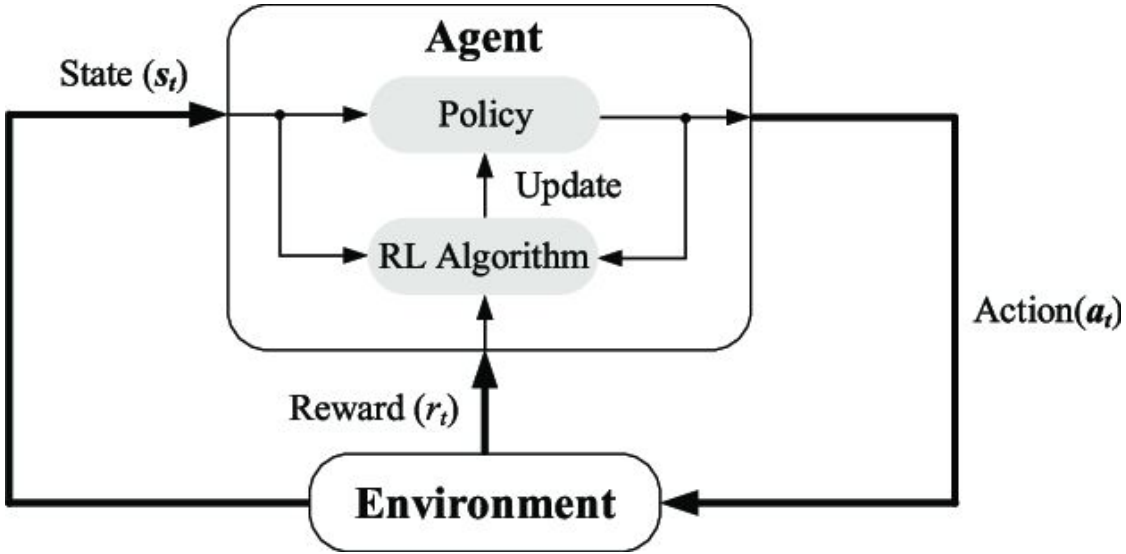
It provides the prompt, previous dialogue, or code problem

Reward = feedback signal

Comes from humans (preferences), AI judges, or verifiers (pass/fail)

ChatGPT is like an RL agent taking a good action and receiving a high reward

What is Reinforcement Learning (RL)? 🧭



Three Reward Regimes 📊

Preference-based (RLHF)

- Humans compare two responses → label A > B
- Captures subjective qualities like clarity, politeness, helpfulness
- Costly but grounds the model in human intent

AI-based (RLAIF)

- A stronger model or rule-based constitution acts as judge
- Scalable, fast, consistent
- Risk: feedback bias or “echo” - model rewards its own habits

Three Reward Regimes 📊

Verifiable (Correctness)

- External checker or test provides pass/fail or numeric score
  - unit tests for code, math solver for proofs, KB for facts
- Dense and unbiased signals → enables self-improvement loops

Recent Trend (2024 -):

Research and industry move toward verifiable rewards as the foundation for reliable reasoning and self-improvements

High-Level Pipeline 🧰

1. Collect Data & Rewards

   Gather prompts, outputs, and feedback signals - human, AI, or verifier

2. Train Reward / Verification Mechanism

   Convert feedback into a reward model or objective checker R(x,y)

3. Optimize Policy (PPO variants)

   Update model to maximize reward while staying close to reference

4. Evaluate & Iterate

   Use human + automated metrics to check alignment and stability

5. Self-Improvement Flywheel

   Use current model `π_t` to generate and judge data → train `π_{t+1}`

Formal Objective (Bird's-Eye) 🧮

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[R(x,y) - \beta \, \mathrm{KL}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x)\big) \right]$$

R(x, y): reward for response y on prompt x

β: KL penalty weight that controls how much the new policy can deviate from the reference

`π_ref`: the reference policy (usually the Supervised Fine-tuned model)

The KL term acts as a stability constraint - it prevents the model from drifting too far from the reference while still learning to maximize rewards

Where Does R(x, y) Come From? 🧪

Preference rewards: R_theta trained on pairwise (A > B)

AI-judge rewards: LLM Autorater

Verifiable rewards: unit tests (code), solvers (math), APIs (tools)

Process rewards: score intermediate reasoning steps

Evaluation at a Glance 🧪

Human eval: side-by-side preference, task-specific rubrics

Automated eval: pass@k

- Run the model's generated code k times with different samples
- pass@1: model must succeed on the 1st try
- pass@k: out of Top k sampled code completions, if ONE passes all test cases → count as success

Example

Task: Write a function is_even(n) that returns True if n is even.

LLM outputs (k = 3):

#1 return n % 2 ❌

#2 return n % 2 == 0 ✅

#3 return n % 2 != 1 ✅

Results:

pass@1 → ❌ (#1 fails)

pass@2 → ✅ (at least one success among 2)

pass@3 → ✅ (at least one success among 3)

Recent Trends in RL for LLMs ⏩

Inexpensive scalable feedback via AI Autoraters

- Anthropic's Constitutional AI show LLM-as-a-Judge can replace costly human raters
- Enables billions of preference pairs at low cost and consistent quality

Better automated verifiers for reasoning & tools

- Rise of verifiable rewards - external solvers, code testers, and symbolic checkers give objective pass/fail signals
- Used in math, code, and scientific reasoning tasks for stable RL updates

Recent Trends in RL for LLMs ⏩

o1 / Claude / Gemini Thinking models

- OpenAI o1 System Card - RL trains models “to reason using chain-of-thought”
- Anthropic’s Claude ‘Think’ Tool focus on process-level reasoning and verification

Convergence: RLHF → RLAIF → Verifiable RL → Self-Improving

- Together these advances mark scalable, verifiable, reasoning-driven RL

Data and Rewards

Three Data Regimes 🏗️

1. Human Preference (HF) - people compare A vs B
2. AI Feedback (AIF) - stronger model or rules judge outputs
3. Verifiable Signals - objective checkers return scores

Human Preference Data 👩‍🧍

Prompt → (Response A, Response B) → Human label A > B

Train a reward model `R_phi` to assign higher scores to preferred answers

Scale ≈ 10^5 – 10^6 pairs in modern datasets

Prompt: “Summarize the Reddit thread about commuting in NYC.”

Response A: *3-sentence concise summary capturing key points*

Response B: *long, repetitive version*

Humans choose A > B → `R_phi` learns human preference

Human Preference Data 👩‍🍼

Pros:

- ✅ Captures genuine human values and intentions
- ✅ Produces high-quality, nuanced judgments (tone, empathy, clarity)

Cons:

- ⚠️ Subjective and labor-intensive
- ⚠️ Drift in annotator style over time
- ⚠️ Slow refresh → less suited for continual updates

AI Feedback & Constitutional Rules 🤖

Use LLMs as a rater/judge for responses

Common methods:

- RLAIF - GPT-5/Claude acts as evaluator for cheaper large-scale labeling
- Constitutional AI - judges responses using written “virtues”
- Self-Rewarding LMs - model critiques its own reasoning and generates preference data without human raters

AI Feedback & Constitutional Rules 🤖

Pros:

- ✅ Scalable - millions of comparisons at minimal cost
- ✅ Consistent - no fatigue or annotator drift
- ✅ Fast refresh - easy to regenerate data for new tasks

Cons:

- ⚠️ Style bias - judge prefers outputs written in its own phrasing
- ⚠️ Echo chamber - feedback loop reinforces same reasoning patterns

Verifiable Rewards ✅

Definition: reward can be checked by an external verifier

Examples:

- Math/Logic → symbolic solver
- Code → unit tests or compiler results
- QA → knowledge base lookup
- Tool use → API return values

Dense and objective → stable training and less bias

Verifiable Rewards ✅

“Anything that is verifiable can be directly optimized with RL.”

Hybrid Data Pipelines 🔄

Mix multiple reward sources, and combine strengths of different feedback types:

- Human preference - captures values and empathy
- AI Feedback (scale) - provides large, consistent data cheaply
- Verifier (correctness) - ensures factual and logical accuracy

Curriculum approach

Start with broad AI feedback for coverage → End with strict verifiable checks for reliability and truth

Goal

Balance breadth in one training loop → diverse input from humans, AI judgment, or objective verification

Data Bias and Quality Issues ⚠️

Human annotator bias (verbosity, tone)

LLM judge bias (favors its own style)

Distribution drift between synthetic and real queries

Quality control practices

- Majority vote: aggregate several judges' opinions for stability
- Reward normalization: rescale scores to avoid runaway values
- Gold human subset: maintain a small, hand-checked dataset to monitor drift

PROMPT
ACTION
REWARD

Core Algorithms for RL in LLMs

Formal Objective (Bird's-Eye) 🧮

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[R(x,y) - \beta \, \mathrm{KL}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x)\big) \right]$$

R(x, y): reward for response y on prompt x

β: KL penalty weight that controls how much the new policy can deviate from the reference

π_ref: the reference policy (usually the Supervised Fine-tuned model)

The KL term acts as a stability constraint - it prevents the model from drifting too far from the reference while still learning to maximize rewards

On-Policy vs Off-Policy 🔁

On-policy:

The same policy that generates data is the one being updated

Examples: PPO, A2C

Pros: stable, matches current distribution

Cons: needs new rollouts each update → expensive for LLMs

On-Policy vs Off-Policy 🔁

Off-policy:

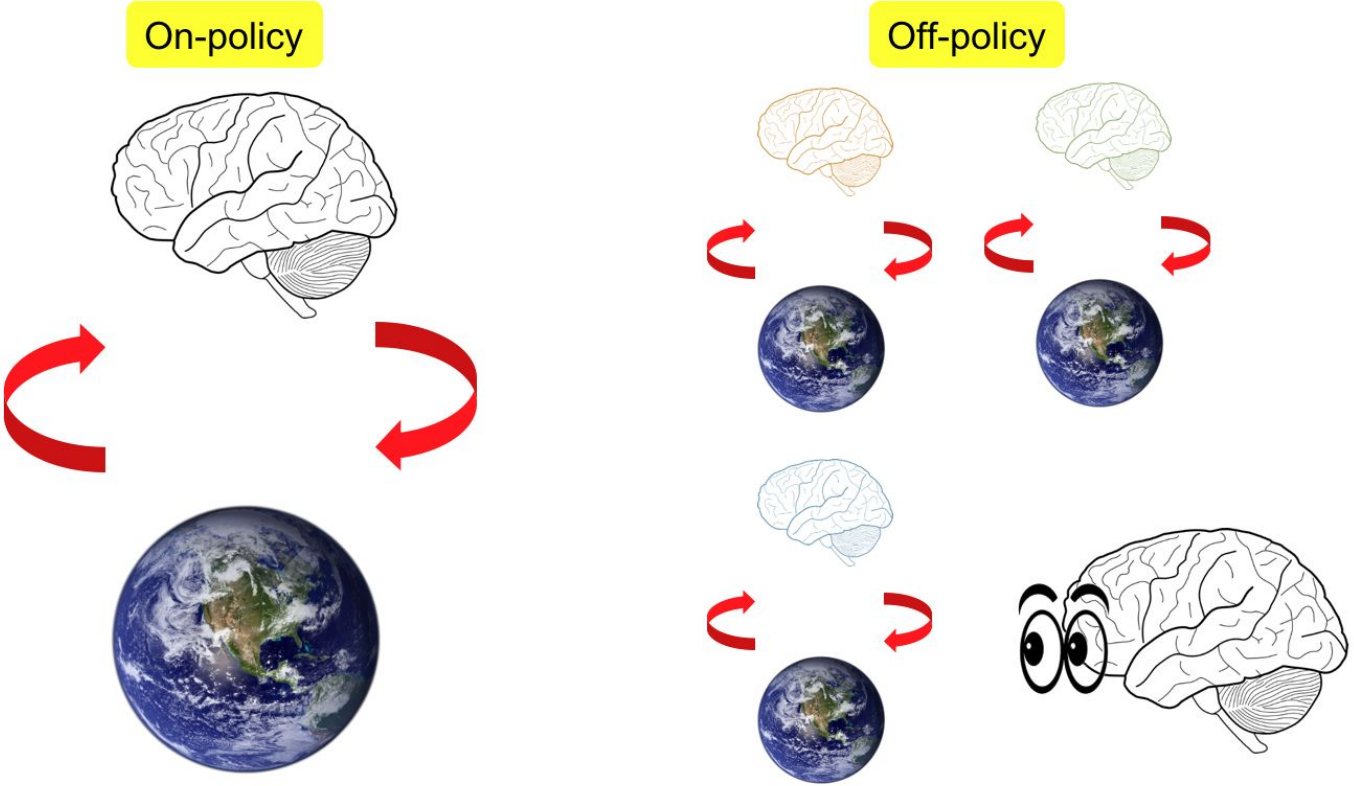
Uses data from other policy models

Examples: DPO, Q-learning

Pros: sample-efficient, can reuse old data

Cons: risk of mismatch between data and current policy

On-Policy vs Off-Policy 🔁



On-Policy vs Off-Policy 🔁

Aspect	On-Policy	Off-Policy
Definition	Learns from data generated by the *current* policy being trained	Learns from data collected by *any* policy (past or other)
Data freshness	Always uses new samples each iteration	Reuses existing or replay-buffer data
Examples (LLMs)	PPO in RLHF	DPO, IPO
Advantages	Matches current behavior → low distribution mismatch; stable learning	Sample-efficient, cheaper, scalable; no need for new rollouts
Disadvantages	Costly - requires generating new responses; slower training	Possible data mismatch; relies on old or static preferences

PPO: Proximal Policy Optimization 🧩

Core idea: small, controlled policy updates

MAXIMIZE objective (simplified version):

$$L(\theta) = \mathbb{E}_t\Big[\mathrm{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\, A_t\Big] \;-\; \beta\,\mathrm{KL}\big(\pi_\theta(a_t \mid s_t) \,\big\|\, \pi_{\mathrm{ref}}(a_t \mid s_t)\big)$$

$$\text{where} \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)}$$

Deep-dive: Advantage A_t 🧩

Intuition: Advantage `A_t` tells us “Is this action better OR worse than random action?”

- `A_t` > 0 → this action was better than expected
- `A_t` < 0 → this action was worse than expected

So PPO tries to:

- increase the probability of actions with positive advantage
- decrease the probability of actions with negative advantage

Toy Example 🎯

Prompt: “Explain what a binary tree is to a beginner.”

Action 1: clear, simple explanation → human/AI gives high score → $A_t \approx +2$

Action 2: overly formal, confusing answer → low score → $A_t \approx -1$

During training:

- For Action 1 → PPO pushes up its probability
- For Action 2 → PPO pushes down its probability

Over time, the model learns to prefer answers with consistently positive A_t

Deep-dive: Clip ✂️

In PPO we have a ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

This says “how much did we change the probability of this action?”

- r_t > 1 → we increased its probability
- r_t < 1 → we decreased its probability

But we do **NOT** want huge jumps, even if A_t is large. That is where clipping comes in:

- We restrict r_t to a range like [1 - ε, 1 + ε], e.g. [0.8, 1.2]
- This implies: “You can change the probability, but not too much in one step.”

Toy Example 🎈

Let $\varepsilon = 0.2 \rightarrow$ allowed range [0.8, 1.2]

Good action: $A_t = +2$

- Suppose $r_t = 1.8$ (big increase)
- Clipped to 1.2 → we still increase its probability, but not crazily

Bad action: $A_t = -1$

- Suppose $r_t = 0.4$ (huge decrease)
- Clipped to 0.8 → we still decrease its probability, but in a controlled way

Key idea for clipping:

> Let us learn from advantage… but do NOT overreact in a single update

Deep-dive: KL divergence 🧩

$$\mathrm{KL}(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid s_t)) = \sum_a \pi_\theta(a \mid s_t) \, \log \frac{\pi_\theta(a \mid s_t)}{\pi_{\mathrm{ref}}(a \mid s_t)}$$

measures how different the new policy is from a reference policy (old model or SFT model)

- Small KL → behavior is similar to reference
- Large KL → behavior has drifted far away

In practice, we subtract β * KL from the objective:

If KL becomes too big, the loss punishes the model

This pushes the new policy back toward the reference

Putting It Together 🌈

Advantage `A_t`

- Says whether a particular answer was better or worse than usual
- Good A_t → probability up; bad A_t → probability down

Clipping

- Stops probability changes from being too extreme
- Keeps learning stable and prevents “one-batch disasters”

KL term

- Keeps the new model close to the reference model
- Preserves style, safety, and general behavior while optimizing reward

💡 PPO uses 1) advantage to know which actions to favor, 2) clipping to avoid over-updating, and 3) KL as a safety belt to keep the policy model from drifting too far while it learns

PPO in LLM Training 🧠

1. Sample prompts from dataset
2. Generate responses using current policy π
3. Get rewards from reward model
4. Update via PPO loss with rewards
5. Evaluate and repeat

Retrieval Augmented Generation

LLM as a Retriever 🔍

So far we have focused on training the model to make it smarter:

- Pre-training → learn language
- SFT → learn to follow instructions
- RL → learn to optimize for quality, safety, reasoning

But even the best-trained model has limits:

- Knowledge is frozen at training cutoff
- Confident but wrong → hallucination

What if instead of memorizing everything, the model could look things up? → Retrieval-Augmented Generation

RL teaches the model how to think, and RAG teaches the model where to look

RAG Pipeline Overview

Retrieval-Augmented Generation (RAG) == LLM + Retrieval system

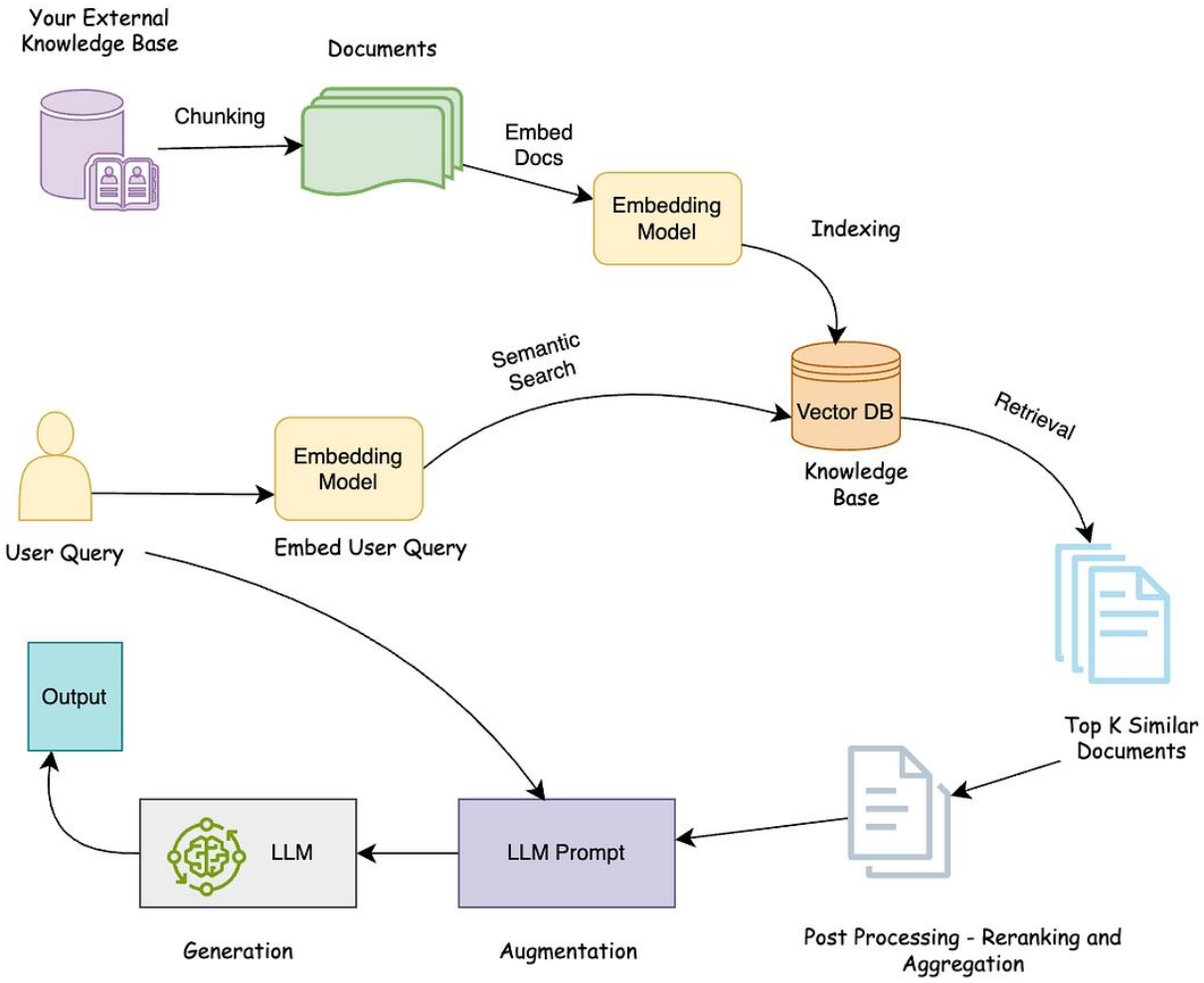
Query → Embed → Retrieve Top-K → LLM

Query - The user provides a question

Embed - Convert the query into a vector representation using an embedding model

Retrieve Top-K - Search a vector store to find the most relevant documents based on semantic similarity

LLM - The retrieved context is injected into the prompt, enabling the model to produce grounded responses


Your External Knowledge Base
Documents
Chunking
Embed Docs
Embedding Model
Indexing
Semantic Search
Vector DB
Knowledge Base
Retrieval
Embedding Model
User Query
Embed User Query
Output
Top K Similar Documents
LLM
LLM Prompt
Generation
Augmentation
Post Processing - Reranking and Aggregation

How Are Embeddings Trained?

Supervised: Train directly on similar vs. dissimilar pairs

Positive pair: (x_a, x□) - semantically similar

Negative pair: (x_a, x□) - semantically different

Pros: maximizes embedding usefulness in retrieval / ranking

Cons: requires curated data

Training Objective: Contrastive Learning

Most modern embedding models = Contrastive Learning

Given a batch of sentences:

- Pull positive pairs close
- Push negative pairs far away

Goal: Learns a geometry where “similar = close, different = far” semantically

Triplet Loss

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\ d(f(x_a), f(x_p)) - d(f(x_a), f(x_n)) + \alpha\right)$$

Goal - Learn an embedding space where:

- Anchor and Positive are close
- Anchor and Negative are far

Input Triplet

- x_a : anchor
- x_p : positive (similar meaning)
- x_n : negative (different meaning)

Triplet Loss

Margin Constraint

```
distance(x_a, x_p) + margin < distance(x_a, x_n)
```

Intuition

- Pull the anchor toward the positive
- Push the anchor away from the negative
- Create a structured embedding space with clear separation

Common Applications

- Semantic similarity
- Retrieval systems

Vector Stores

Popular options:

FAISS (Meta) - in-memory, blazing fast, no managed service

- Best for: research, prototyping, single-machine workloads
- Con: no built-in persistence or metadata filtering

Pinecone (LangChain) - fully managed cloud service, production-grade

- Best for: production apps, auto-scaling, minimal ops overhead
- Con: vendor lock-in, cost at scale

Key tradeoff: control & cost (FAISS) vs convenience & scale (Pinecone)

RAG Evaluation Metrics

Retrieval Quality: did we find the right documents?

- Recall@K - fraction of relevant docs in Top-K results
- MRR (Mean Reciprocal Rank) - how early does the first relevant doc appear?

Generation Faithfulness: is the answer grounded in retrieved context?

- Does the response only use facts from the retrieved docs?
- Hallucination = answer contains claims not supported by any retrieved passage

Answer Relevance: does the answer actually address the question?

- Is the response on-topic, complete, and useful?

⚠️ Common failure modes:

- High retrieval recall but low faithfulness → model ignores context and hallucinates
- High faithfulness but low relevance → model copies context but misses the question

RAG vs Fine-Tuning

2 ways to give an LLM new knowledge, yet when to use which?

RAG

- Knowledge lives externally in a document store
- Update knowledge by updating the corpus — no retraining
- Best for: frequently changing data, private docs, citations needed
- Cost: retrieval infra + embedding compute at query time

Fine-Tuning

- Knowledge is baked into model weights
- Update knowledge by retraining — expensive and slow
- Best for: teaching style, format, domain-specific reasoning patterns

RAG vs Fine-Tuning

When to use what:

- Need fresh or private facts? → RAG
- Need the model to behave differently (tone, format, reasoning)? → Fine-Tuning
- Need both? → Fine-Tune + RAG — fine-tune for style, RAG for facts, aka Agentic AI

💡 Rule of thumb: RAG changes what the model knows, Fine-Tuning changes how the model thinks